

BASIC CURD OPERATIONS

Create the Keyspace (if not already existing)

```
CREATE KEYSPACE IF NOT EXISTS university
WITH REPLICATION = {
  'class' : 'SimpleStrategy',
  'replication_factor' : 3
};
```

Use the Keyspace

```
USE university;
```

Create the Student Table

```
CREATE TABLE student ( student_id INT PRIMARY KEY, name TEXT, age INT, major TEXT,
enrollment_date DATE );
```

Insert 3 Rows into the Table

```
INSERT INTO student (student_id, name, age, major, enrollment_date)
VALUES (1, 'gopi', 20, 'Computer Science', '2023-09-01');
```

```
INSERT INTO student (student_id, name, age, major, enrollment_date)
VALUES (2, 'krishnan', 22, 'Civil Engineering', '2022-09-01');
```

```
INSERT INTO student (student_id, name, age, major, enrollment_date) VALUES (4, 'aadhav', 21,
'Mathematics', '2023-01-15');
```

Read - Query all records

```
SELECT * FROM student;
```

Read - Query a specific record

```
SELECT * FROM student WHERE student_id = 1;
```

Update - Modify a student's major

```
UPDATE student SET major = 'Data Science' WHERE student_id = 1;
```

Update - Modify a student's age

```
UPDATE student SET age = 23 WHERE student_id = 2;
```

Delete - Remove a specific record

```
DELETE FROM student WHERE student_id = 3;
```

Delete - Remove a specific column for a student

```
DELETE major FROM student WHERE student_id = 1;
```

Delete - Remove all records from the table

```
TRUNCATE student;
```

DDL QUERIES

Select DB

```
USE University;
```

Alter Keyspace (DB)

```
ALTER KEYSPACE University WITH REPLICATION = { 'class' : 'NetworkTopologyStrategy',  
'datacenter1' : 1 };
```

```
ALTER KEYSPACE University WITH REPLICATION = { 'class' : 'NetworkTopologyStrategy',  
'datacenter1' : 1 } AND DURABLE_WRITES = false;
```

Delete DB

```
DROP KEYSPACE University;
```

Alter Table

Rename

Cannot rename non-PRIMARY KEY column

```
ALTER TABLE student RENAME student_id to ssid; //valid
```

```
ALTER TABLE student RENAME name to sname; //invalid
```

Change datatype of a column

Does not support changing a column's data type

```
ALTER TABLE student ALTER age TYPE double; //invalid
```

Adding new Column

```
ALTER TABLE student ADD mark INT;
```

Update default values for new columns

```
UPDATE student SET mark = 85 WHERE student_id = 1;
```

```
UPDATE student SET mark = 90 WHERE student_id = 2;
```

```
UPDATE student SET mark = 78 WHERE student_id = 3;
```

Update to all students at a time is invalid.

Removing Existing Column

```
ALTER TABLE student DROP enrollment_date;
```

To drop table

```
DROP TABLE student;
```

Aggregate Functions:

Can apply only on numerical column

Calculate the Total Sum of Marks

```
SELECT SUM(mark) AS total_marks FROM student;
```

Calculate the Average Marks

```
SELECT AVG(mark) AS average_marks FROM student;
```

Find the Maximum Marks

```
SELECT MAX(mark) AS max_marks FROM student;
```

Find the Minimum Marks

```
SELECT MIN(mark) AS min_marks FROM student;
```

Count the Number of Students with Marks

```
SELECT COUNT(mark) AS count_marks FROM student;
```

WHERE Clauses in CQL:

WHERE condition on primary key column:

For Equality:

```
SELECT * FROM student WHERE ssid = 2;
```

For <, >:

```
SELECT * FROM student WHERE token(ssid) > token(2);
```

WHERE condition on Secondary key column:

```
CREATE TABLE IF NOT EXISTS flag1(sid INT, age INT, name TEXT, PRIMARY KEY (sid, age));
```

//here sid is primary key, age is secondary key. Both combined is Partial keys

Partial keys: Sid can accept duplicate; age can accept duplicate but Sid and age both cannot accept duplicate

For Equality:

```
SELECT * FROM flag1 WHERE age = 25 ALLOW FILTERING;
```

For <, >

```
SELECT * FROM flag1 WHERE age < 30 ALLOW FILTERING;
```

WHERE condition on NON key column:

For Equality Condition

Create index in WHERE condition field

```
CREATE INDEX ON student(mark);
SELECT * FROM student WHERE mark = 90;
```

For <,>:

Not possible. Possible only when a condition is added on primary key as well.

```
SELECT * FROM student WHERE mark < 90; //INVALID
SELECT * FROM student WHERE mark < 90 ALLOW FILTERING; //INVALID
SELECT * FROM student WHERE token(mark) < token(90); //INVALID
```

```
CREATE TABLE IF NOT EXISTS flag2 (sid INT, mark INT, name TEXT, PRIMARY KEY (sid,
mark) );
SELECT * FROM flag2 WHERE mark < 90 ALLOW FILTERING; //valid
```

WHERE with Multiple Conditions (AND)

If one condition on primary and another on secondary:

For Equality

```
SELECT * FROM flag2 WHERE sid = 2 AND mark = 90;
```

For <,>

```
SELECT * FROM flag2 WHERE sid = 2 AND mark > 90; //VALID
SELECT * FROM flag2 WHERE token(sid) > token(2) AND mark = 90 ALLOW FILTERING;
SELECT * FROM flag2 WHERE token(sid) > token(2) AND mark > 90 ALLOW FILTERING;
```

If one condition on primary and another on non-key column:

For Equality

```
CREATE INDEX ON flag2(name);
SELECT * FROM flag2 WHERE mark = 90 AND name = 'gopi' ALLOW FILTERING;
SELECT * FROM flag2 WHERE sid = 90 AND name = 'gopi' ALLOW FILTERING;
```

For <,>

```
SELECT * FROM flag2 WHERE mark > 90 AND name = 'gopi' ALLOW FILTERING;
SELECT * FROM flag2 WHERE token(sid) > token(90) AND name = 'gopi' ALLOW
FILTERING;
```

Not possible to apply < > on NON Key fields

WHERE with Multiple Conditions (OR)

OR Not possible in WHERE Conditions.

IN Clause for Multiple Values //IN cannot work on NON PRIMARY KEY FIELD

```
SELECT * FROM student WHERE sid IN (1, 2);
```

Updates in CQL:

Replacing existing values:

```
UPDATE student SET mark = 85 WHERE ssid = 1;
```

Adding values to a Column Value (Indirect Method)

```
SELECT mark FROM student WHERE student_id = 1;  
UPDATE student SET mark = 160 WHERE student_id = 1;
```

Update based on condition:

Not possible to update multiple rows at a time.

```
UPDATE student SET mark = 45 WHERE student_id > 2 ALLOW FILTERING; //INVALID
```

```
UPDATE student SET mark = 45 WHERE token(student_id) > token(2); //INVALID
```

BATCH Statements

```
BEGIN BATCH  
    INSERT INTO student (student_id, name, age, major, enrollment_date) VALUES (1, 'gopi',  
    20, 'Computer Science', '2023-09-01');  
    INSERT INTO student (student_id, name, age, major, enrollment_date) VALUES (2,  
    'krishnan', 22, 'Civil Engineering', '2022-09-01');  
    SELECT * FROM student;  
APPLY BATCH;
```

Cassandra using python:

Update marks by 10%

Update.py

```
from cassandra.cluster import Cluster  
cluster = Cluster(['127.0.0.1']) # Replace with your Cassandra node IPs  
session = cluster.connect('university')  
rows = session.execute("SELECT sid, name, mark, cgpa FROM student3;")  
for row in rows:  
    new_mark = int(row.mark * 1.1) # Increase mark by 10%  
    session.execute(f"DELETE FROM student3 WHERE sid = {row.sid} AND mark = {row.mark};")  
    session.execute(f"INSERT INTO student3 (sid, name, mark, cgpa) VALUES ({row.sid},  
'{row.name}', {new_mark}, {row.cgpa});")  
cluster.shutdown()
```

Collections in CQL:

Creating table with Collections:

```
CREATE TABLE student1 (  
  sid INT PRIMARY KEY,      -- Primary Key (student ID)  
  name TEXT,                -- Student Name  
  marks LIST<INT>,          -- List of Marks  
  courses_id SET<TEXT>,     -- Set of Course IDs (using TEXT)  
  address MAP<TEXT, TEXT>   -- Map for Address (e.g., {'city': 'Chennai', 'state': 'Tamil  
  Nadu'})  
);
```

Inserting values for collection type columns:

```
INSERT INTO student1 (sid, name, marks, courses_id, address) VALUES ( 1, 'Alice Johnson',  
[85, 90, 95], {'C101', 'C102'}, {'city': 'Chennai', 'state': 'Tamil Nadu'});
```

```
INSERT INTO student1 (sid, name, marks, courses_id, address) VALUES ( 2, 'Bob Smith', [88,  
76, 91], {'C201', 'C202'}, {'city': 'Mumbai', 'state': 'Maharashtra'} );
```

```
INSERT INTO student1 (sid, name, marks, courses_id, address) VALUES ( 3, 'Carol Davis',  
[72, 84, 79], {'C301', 'C302'}, {'city': 'Bengaluru', 'state': 'Karnataka'} );
```

Retrieve all the values:

```
SELECT * FROM student1;
```

Updates:

```
UPDATE student1 SET courses_id = courses_id + {'C103'} WHERE sid = 1;  
UPDATE student1 SET courses_id = courses_id - {'C102'} WHERE sid = 1;  
UPDATE student1 SET address['state'] = 'Tamil Nadu' WHERE sid = 1;  
UPDATE student1 SET address['zip'] = '533434' WHERE sid = 1;
```

Retrieve:

```
Retrieve All Elements from the 'marks' List for a Student  
SELECT marks FROM student1 WHERE sid = 1;
```

Check If the 'marks' List Contains a Specific Value (e.g., 90)

```
CREATE INDEX IF NOT EXISTS marks_idx ON student1(marks);  
SELECT * FROM student1 WHERE marks CONTAINS 90;
```

Retrieve the index-based values from collection column is invalid

```
SELECT marks[0] AS first_mark FROM student1 WHERE sid = 1; //not possible
```

Retrieve a Map Based values

```
SELECT * FROM student1 WHERE address['city'] = 'Chennai'; //not possible
```

CQL User-defined Data Type

Create User defined datatype:

```
CREATE TYPE address ( sid int, street text, city text, zip int );
```

DDL for user defined datatypes:

```
DESCRIBE TYPE address;  
DESCRIBE TYPES;  
DROP TYPE address;
```

Create table using User defined datatype:

```
CREATE TABLE student3 ( sid int PRIMARY KEY, name text, address frozen<address>, mark  
int);
```

Inserting values for user defined columns:

```
INSERT INTO student3 JSON '{"sid":434, "address":{"sid":434,"street":"65A  
Block","city":"goa","zip":522236}}';
```

```
INSERT INTO student3 (sid, address) VALUES (234, {"sid":234,"street":"65A  
Block","city":"goa","zip":522236});
```

Retrieve all values:

```
SELECT address FROM student3 WHERE sid=434;
```

Retrieve only one user defined field:

```
SELECT address.city FROM student3 WHERE sid=434;
```

Applying WHERE Conditions (Exact address match):

```
CREATE INDEX id_add ON student3(address);  
SELECT * FROM student3 WHERE address={sid: 434, street: '65A Block', city: 'goa', zip:  
522236};
```

Cassandra does not allow direct filtering of individual fields within the frozen UDT.

```
SELECT * FROM student3 WHERE address.city='Goa'; //INVALID
```

Limitations:

- **No Indexes on Counter Columns:** You can't create secondary indexes on counter columns.
- **No Primary Keys for Counter Columns:** You cannot set counter columns as primary keys or include them in composite primary keys.
- **No Primary Key Modification:** Primary key definitions cannot be altered after table creation. You must drop and recreate the table to change primary keys.
- **No JOIN Operations:** Cassandra does not support SQL-style JOIN operations. You need to denormalize data or perform manual joins in the application layer.
- **No OR Conditions in WHERE Clauses:** Cassandra does not allow the use of OR in WHERE clauses, so you need to use multiple queries or restructure the table.
- **No Range Queries on Secondary Indexes:** Cassandra's **secondary indexes** do not support range queries (e.g., $>$, $<$, $>=$, $<=$). They are designed primarily for equality-based filtering (e.g., `cgpa = 8`).
- **Range Updates Are Not Allowed:** Cassandra does not support **range conditions** (e.g., $>$, $<$, $>=$, $<=$) in the WHERE clause for updating rows.
- **Ordering Constraints:** Cassandra's ORDER BY can only be used when: The **partition key** is specified in the WHERE clause with an equality condition. The column used for ordering is a **clustering column**. **Global sorting** across all partitions is not possible with native Cassandra queries.
- **token() Function Limitation:** The token() function can only be applied to **partition keys** in the WHERE clause. Using token() with non-partition key columns (e.g., `token(cgpa)`) is invalid.
- **ALLOW FILTERING Performance Issues:** Using ALLOW FILTERING can enable range queries or filtering on non-key columns, but it often results in a **full table scan**, making it highly inefficient for large datasets.
- **Limited Data Types in Primary Keys:** The primary key can only contain certain data types like text, int, UUID, etc. Complex types (e.g., collections) cannot be used in primary keys.
- **No DISTINCT Without Partition Key:** Using SELECT DISTINCT requires specifying a partition key; otherwise, it won't work.
- **Inefficient Range Queries on Wide Rows:** Range queries ($<$, $>$, $<=$, $>=$) work only on clustering columns and can be slow if not properly indexed or ordered. (you can use token function)
- **Partition Key Restriction:** You cannot perform a WHERE clause filter without specifying at least part of the partition key.
- **No Aggregation Functions Beyond Basic Ones:** Cassandra supports basic aggregation like COUNT, SUM, etc., but more advanced aggregations require third-party tools like Spark or manual aggregation in the application.
- **Limited IN Clauses:** Using IN in WHERE clauses works but can lead to inconsistent performance and potential timeouts, especially with a large number of values.
- **Limit on Batching:** Batch statements should be used sparingly, only for operations involving multiple rows with the same partition key; otherwise, they can lead to inconsistencies.
- **Limit on Collection Size:** Collections (sets, lists, maps) have a size limit of 64 KB per column, making them unsuitable for very large collections of data.
- **No Direct Indexing of Map/Set Values:** Only keys in maps can be indexed, but not the values. Sets and lists are not directly indexable either.